Writing sample 2 of 4: Concept page. Prepared by Judah Paemka for the Grafana Labs Staff Technical Writer application, October 2026. Original work written to Grafana Writers' Toolkit conventions and checked with the Grafana Vale style rules; not published Grafana documentation. Companion to Sample 3: the same LLM platform problems, rewritten for a developer audience. The Alloy configuration was validated on Alloy v1.21.0, and the metric names and PromQL were checked against live data in Prometheus 3.15.0.

PAGE FRONT MATTER (HUGO, DOCS-AS-CODE SOURCE)

```
title: Observability for LLM applications
menuTitle: LLM applications
description: Learn which metrics, traces, and logs to collect from applications that call large
language models, how the OpenTelemetry GenAI semantic conventions name them, and how to route them
to Grafana with Grafana Alloy.
weight: 200
keywords:
  - LLM
  - generative AI
  - OpenTelemetry
  - Alloy
  - traces
  - metrics
```

Observability for LLM applications

An application that calls a large language model (LLM) can fail in ways that standard request monitoring misses. A request can return HTTP 200 with a response your code can't use. A retry inside one request can double its cost. A streamed answer can feel slow even when total latency looks normal.

This page explains what to measure in an LLM application and which signal to use for each one. It also shows how to send that data to Grafana with Grafana Alloy. It assumes you know the basics of metrics, logs, and traces.

How LLM applications fail differently

The following table lists common failures and why standard HTTP monitoring tends to miss them.

Failure	What users notice	Why standard signals miss it
Slow first token	The answer takes seconds to start.	Request duration covers the whole stream, so the total hides a slow start.
Unusable structured output	A feature breaks or shows an error.	The model call returns HTTP 200. The problem is in the content.
Hidden retries and repairs	Answers are slower and cost more.	The extra model calls happen inside one user request.

Failure	What users notice	Why standard signals miss it
Rejected tool calls	The answer is incomplete.	The application handles the rejection, so no error appears.
Token cost drift	The monthly bill grows.	Latency and error dashboards don't show token counts.

These failures often appear together. When a model returns output that fails validation, a repair step calls the model again. That adds both time and tokens to the same request.

Choose a signal for each question

Each signal answers a different kind of question.

Question	Signal	Store in
Where did the time go in this one slow request?	Traces	Grafana Tempo
Is latency, cost, or the validation failure rate getting worse across all requests?	Metrics	Prometheus or Grafana Mimir
Why did this response fail validation, and what did the application do next?	Logs	Grafana Loki

Traces

One user request often makes several calls: retrieval, model calls, tool calls, and validation. A trace records each call as a span, so you can see which step was slow and which one failed.

The OpenTelemetry GenAI semantic conventions suggest a span name of `{gen_ai.operation.name} {gen_ai.request.model}`, for example `chat model-a`. Add attributes such as `gen_ai.provider.name`, `gen_ai.request.model`, and `error.type` to each span. Then you can filter traces by provider, model, and type of failure.

Metrics

Use metrics to answer questions about all traffic and to drive alerts. LLM applications need two kinds:

- **Standard GenAI metrics:** your library emits these for you. They include model call duration, time to the first chunk of a streamed response, and token usage.
- **Application quality metrics:** you define these yourself, because no library knows your output schema or your business rules. For example, a counter named `app.llm.response.validations` with an `outcome` attribute of `valid`, `repaired`, or `rejected`

shows how often the model returns output your code can use.

Logs

Use logs for detail that doesn't fit in a metric. Examples include the validation error message, the name of a rejected tool, or the reason for a retry. Include the trace ID in each log line so you can jump from a log line to the full trace.

> **CAUTION**
>
> Prompts and model responses can contain personal or confidential data. In the GenAI semantic conventions, message content capture is opt-in. It uses attributes such as `gen_ai.input.messages` and `gen_ai.output.messages`. Turn it on only where your data policy allows it. Where you can, record sizes, counts, and validation results instead of raw content.

Names for generative AI metrics

The OpenTelemetry GenAI semantic conventions have Development status, so names can change between versions. The conventions moved to their own repository, semantic-conventions-genai.

Some names have already changed. Earlier releases defined `gen_ai.client.operation.duration` and one `gen_ai.client.token.usage` histogram for model calls. The current draft calls model calls inference. It reports them with the following metrics, and it says not to report `gen_ai.client.operation.duration` for inference.

Metric	Type	Unit	What it measures
`gen_ai.client.inference.duration`	Histogram	`s`	Time from the model request until the full response arrives or the call fails
`gen_ai.client.inference.time_to_first_chunk`	Histogram	`s`	Time until the first chunk of a streamed response arrives
`gen_ai.client.inference.usage.input_tokens`	Counter	`{token}`	Input tokens, including cached tokens
`gen_ai.client.inference.usage.output_tokens`	Counter	`{token}`	Output tokens, including reasoning tokens

Before you build dashboards or alerts, check which names your library emits. In Grafana, open **Explore**, select your metrics data source, and search the metrics browser for `gen_ai`.

Route LLM telemetry to Grafana with Alloy

Most LLM instrumentation libraries send data in OTLP, the OpenTelemetry Protocol. Alloy can receive OTLP and send each signal to the right backend. Metrics go to Mimir, logs go to Loki, and traces go to Tempo.

```
// Receive OTLP from the instrumented application.
otelcol.receiver.otlp "default" {
  grpc {}
  http {}

  output {
    metrics = [otelcol.processor.batch.default.input]
    logs    = [otelcol.processor.batch.default.input]
    traces  = [otelcol.processor.batch.default.input]
  }
}

// Batch telemetry before export to reduce outbound requests.
otelcol.processor.batch "default" {
  output {
    metrics = [otelcol.exporter.prometheus.default.input]
    logs    = [otelcol.exporter.otlphttp.loki.input]
    traces  = [otelcol.exporter.otlp.tempo.input]
  }
}

// Metrics: convert to Prometheus format and remote write to Mimir.
otelcol.exporter.prometheus "default" {
  forward_to = [prometheus.remote_write.mimir.receiver]
}

prometheus.remote_write "mimir" {
  endpoint {
    url = "http://mimir:9009/api/v1/push"
  }
}

// Logs: send to the Loki native OTLP endpoint.
otelcol.exporter.otlphttp "loki" {
  client {
    endpoint = "http://loki:3100/otlp"
  }
}

// Traces: send to Tempo over OTLP gRPC.
otelcol.exporter.otlp "tempo" {
  client {
    endpoint = "tempo:4317"
    tls {
      insecure = true
```

```
    }
  }
}
```

By default, `otelcol.receiver.otlp` listens for OTLP over gRPC on port `4317` and over HTTP on port `4318`. Point your application at Alloy. For example, set the standard environment variable `OTEL_EXPORTER_OTLP_ENDPOINT=http://alloy:4318`.

Replace the `mimir`, `loki`, and `tempo` host names with the addresses of your own backends. The `tls { insecure = true }` block turns off TLS for the connection to Tempo. Use it only on a trusted local network.

How metric names change in Prometheus

When `otelcol.exporter.prometheus` converts OTLP metrics, it replaces dots with underscores. It also adds unit and type suffixes. Attributes become labels in the same way.

OpenTelemetry name	Prometheus name
`gen_ai.client.inference.duration`	`gen_ai_client_inference_duration_seconds`
`gen_ai.client.inference.usage.input_tokens`	`gen_ai_client_inference_usage_input_tokens_total`
`app.llm.response.validations`	`app_llm_response_validations_total`
`gen_ai.request.model` attribute	`gen_ai_request_model` label

Example queries

Use the following PromQL queries as a starting point for dashboard panels and alerts.

95th percentile model call duration, by model:

```
histogram_quantile(0.95, sum by (le, gen_ai_request_model)
(rate(gen_ai_client_inference_duration_seconds_bucket[5m])))
```

95th percentile time to first chunk, by model:

```
histogram_quantile(0.95, sum by (le, gen_ai_request_model)
(rate(gen_ai_client_inference_time_to_first_chunk_seconds_bucket[5m])))
```

Input tokens per second, by model:

```
sum by (gen_ai_request_model) (rate(gen_ai_client_inference_usage_input_tokens_total[5m]))
```

Share of responses rejected after validation, by model:

```
sum by (gen_ai_request_model) (rate(app_llm_response_validations_total{outcome="rejected"}[5m]))
/
sum by (gen_ai_request_model) (rate(app_llm_response_validations_total[5m]))
```

Percentile queries are only as precise as the histogram buckets. The semantic conventions suggest fixed bucket boundaries for the duration metrics, from 0.01 seconds up to about 82 seconds. If your library uses its default buckets, check that they cover the latency range of your models.

Choose attributes and labels with care

Every unique set of label values creates a separate time series in Prometheus or Mimir. So the attributes you put on metrics affect cost and query speed.

These attributes work well as metric labels, because each has a small, fixed set of values:

- `gen_ai.provider.name`, `gen_ai.request.model`, and `gen_ai.operation.name`
- `error.type`
- Your own bounded values, such as `outcome`

Keep these off metrics, because each one can have thousands or millions of values:

- User, conversation, and request IDs
- Prompt text and tool arguments

Record high-cardinality values on spans or in log lines instead. Tempo and Loki handle that level of detail, and the trace ID links them to your metrics.

Next steps

- To practice choosing labels for logs, work through *Collect and label JSON application logs with Grafana Alloy*, the companion tutorial to this page.
- Read the `otelcol.receiver.otlp` reference for all receiver options.
- Learn how to link metrics to traces with exemplars.
- Follow changes to the GenAI semantic conventions before you upgrade your libraries.